LIBRARY AND RESOURCE MANAGEMENT SYSTEM

CEP Project Report | Spring 2026

NED University of Engineering and Technology

Course: **Object Oriented Program(CEP)**, Spring 2026
Department: Computer Systems Engineering
Section: B
CEP Group ID: **G3-9**
Submitted To: **Miss Fauzia Yasir**
Date: May 4, 2025

Group Members

Sr.	Full Name	Roll Number
1	Syed Muhammad Irtiza Abbas Rizvi	CS-25089
2	Muhib Ur Rub Qasmi	CS-25086
3	Syed Zayan Mehmood	CS-25090
4	Muhammad Hammad Ul Saboor	CS-25080

1. Introduction

We made a Library and Resource Management System for our CEP project. The program is written in C++ and runs in the console. It has two types of users. A normal user can register, log in, borrow books, return books, and see their history. An admin can add new books, remove books, update book details, and see reports of all users. All data is saved in text files so nothing is lost when the program is closed.

We used many OOP concepts in this project that we studied this semester. These include classes, inheritance, encapsulation, polymorphism, operator overloading, exception handling, and file handling. All of these are explained in this report.

2. All Classes, Attributes, Methods, and Relationships

Class 1: LibraryException (and Sub-Exception Classes)

Protected Attribute:

- message (string) stores the error message text

Methods:

- LibraryException(string msg) constructor that sets the message
- what() returns the error message as a C-style string

Sub-Classes that inherit from LibraryException:

- ResourceNotAvailableException
- InsufficientBalanceException
- LimitExceededException
- RecordNotFoundException
- InvalidDateException

Relationship: All these sub-classes inherit from LibraryException. LibraryException itself inherits from the C++ standard exception class.

Class 2: Resource

Private Attributes:

- id (int)
- title (string)
- author (string)
- category (string)
- available (bool)

Methods:

- Resource() default constructor
- Resource(int, string, string, string, bool) parameterized constructor
- getId(), getTitle(), getAuthor(), getCategory(), getAvailable() getter functions
- isAvailable() returns true if the book is on the shelf

- setAvailable(bool) changes availability
- display() prints all book details on screen
- operator== overloaded to compare two resources by ID
- operator<< overloaded to print resource details using cout

Relationship: Many Resource objects are stored inside the library class using a vector.

Class 3: BorrowRecord

Private Attributes:

- recordId (int)
- resourceId (int)
- username (string)
- resourceTitle (string)
- issueDate (int)
- returnDate (int)
- dueDate (int)
- fineAmount (double)

Methods:

- BorrowRecord() default constructor
- BorrowRecord(int, int, string, string, int, int, int, double) parameterized constructor
- All getter functions for each attribute
- display() prints record details on screen
- returnResource(int, double) processes the return and calculates fine
- isActive() returns true if the book has not been returned yet
- isOverdue() returns true if the due date has passed

Relationship: Many BorrowRecord objects are stored inside the library class using a vector.

Class 4: Person (Abstract Base Class)

Protected Attributes:

- username (string)
- firstName (string)
- lastName (string)
- password (string)

Methods:

- Person() default constructor
- Person(string, string, string, string) parameterized constructor
- getUsername(), getFirstName(), getLastName(), getPassword() getter functions
- virtual void display() = 0 pure virtual function, makes Person abstract
- virtual ~Person() virtual destructor

Relationship: User and Admin both inherit from Person.

Class 5: User (Inherits from Person)

Private Attributes:

- address (string)
- balance (double)

Methods:

- User() default constructor
- User(string, string, string, string, string, double) parameterized constructor
- getAddress(), getBalance() getter functions
- setAddress(), setBalance() setter functions
- addBalance(double) adds amount to balance
- deductBalance(double) removes amount from balance
- display() override of Person's pure virtual function

Relationship: Inherits from Person. Many User objects are stored in library using a vector.

Class 6: Admin (Inherits from Person)

No new private attributes. Uses the ones inherited from Person.

Methods:

- Admin() default constructor
- Admin(string, string, string, string) parameterized constructor
- display() override of Person's pure virtual function

Relationship: Inherits from Person. One Admin object is stored inside the library class.

Class 7: library (Main Controller Class)

Private Attributes:

- resources (vector<Resource>)
- borrowRecords (vector<BorrowRecord>)
- users (vector<User>)
- admin (Admin)
- nextResourceId (int)
- nextRecordId (int)
- finePerDay (double)
- maxBorrowDays (int)

Methods (Public):

- library() constructor, loads all data from files when program starts
- saveAll() and loadAll() handles reading and writing to text files
- registerUser(...) registers a new user after checking all inputs
- loginUser(...) and loginAdmin(...) handles login for user and admin
- borrowResource(...) handles the borrowing process with many validations
- returnResource(...) handles returning a book and calculates any fine
- searchResources(...) finds books by title, author, or category
- displayResources(), displayAllResources(), displayBorrowRecords() display functions

- addResource(...), removeResource(...), updateResource(...) admin functions
- printAllCustomersReports(), printIssuedOverdueReport(...) admin report functions
- updateUserBalance(...) adds balance to a user account
- mainMenu(), userMenu(int), adminMenu() the three main menus of the program

Relationship: This class contains all other classes. It is the main class that runs the whole system.

3. Class Descriptions

LibraryException and Sub-Exception Classes

This group of classes handles all the errors in our system. Instead of just printing an error message and stopping, we made custom exception classes that can be thrown when something goes wrong. For example if a book is not available or if the user has no balance. Each sub-class has its own error message that clearly tells the user what the problem is.

Resource Class

The Resource class represents one book or item in the library. It stores all the information about a book like its ID, title, author, category, and whether it is available or not. It also has two overloaded operators to make comparing and printing easier.

BorrowRecord Class

The BorrowRecord class keeps track of every borrow and return that happens in the system. It saves who borrowed which book, when it was issued, when it is due, and whether it has been returned or not. It is also responsible for calculating the fine when a book is returned late.

Person Class

Person is a base class that holds the common information shared by both Users and Admins, like username, first name, last name, and password. It cannot be used directly to make an object because it has a pure virtual function. This forces every class that inherits from it to define its own version of the display function.

User Class

The User class represents a person who is registered in the library system. It gets everything from Person and also has an address and a balance. The balance is important because fines are taken from it when a book is returned late. All data is private and can only be accessed through getter and setter functions.

Admin Class

The Admin class represents the library administrator. It inherits from Person and overrides the display function. The admin has special powers in the system like adding books, removing books, updating book details, and viewing full reports of all customers.

library Class

The library class is the most important class in our project. It stores all the data including users, books, and borrow records. It has all the logic for borrowing, returning, searching, and managing books. It also runs the main menus for user and admin and handles saving and loading data from files.

4. OOP Concepts Used in This Project

4.1 Classes and Objects

We made 7 classes in our project: Resource, BorrowRecord, Person, User, Admin, library, and LibraryException. Each class groups related data and functions together. For example when a user logs in, the library class uses User and BorrowRecord objects to track what is happening in the session.

4.2 Encapsulation

All data members in our classes are declared as private so they cannot be changed directly from outside the class. We use public getter and setter functions to access them. For example the balance in the User class is private and can only be changed through addBalance() or deductBalance() which makes sure the balance is always updated correctly.

4.3 Inheritance

We used inheritance in two places. First, both User and Admin inherit from the base class Person, so they automatically get the common attributes like username and password without writing them again. Second, all our custom exception classes like LimitExceededException and InsufficientBalanceException inherit from LibraryException, which itself inherits from the standard C++ exception class.

4.4 Abstraction

The Person class is abstract because it has the pure virtual function virtual void display() = 0. This means you cannot create a Person object directly. You can only create a User or Admin object. This is abstraction because it forces us to always work with a specific type instead of a generic one.

4.5 Polymorphism

The display() function is declared as virtual in Person and is overridden differently in User and Admin. When we call display() on a User object it shows the user's balance and address. When called on an Admin object it shows the admin's name and role. This is runtime polymorphism because the right version of the function is picked based on the actual object type.

4.6 Operator Overloading

In the Resource class we overloaded the == operator so we can compare two Resource objects by their ID. This is used in the removeResource() function to find and delete the right book. We also overloaded the << operator so we can use cout on a Resource object directly to print all its details, which makes the code shorter and easier to read.

4.7 Exception Handling

We made a set of custom exception classes that all inherit from LibraryException. In functions like borrowResource() and returnResource() we use throw to raise an error when something goes wrong, like a book not being available or the borrow limit being reached. In the menu functions we wrap these calls in try-catch blocks so the program handles errors properly and does not crash.

4.8 File Handling

We used ifstream and ofstream to read from and write to three text files: users.txt, resources.txt, and records.txt. When the program starts, loadAll() reads all the saved data. After any action like registering, borrowing, or returning a book, saveAll() writes everything back to the files. This keeps the data safe even after the program is closed.

5. Screenshots

Below are screenshots taken while the program was running:

[Screenshot 1: Main Menu on screen]

```
**************************************************
          MAIN MENU - LIBRARY SYSTEM
**************************************************
 1. Register New User
 2. User Login
 3. Admin Login
 4. Exit System

Enter your choice:
```

[Screenshot 2: User registration and login]

```
*****************************************************
          MAIN MENU - LIBRARY SYSTEM
*****************************************************
 1. Register New User
 2. User Login
 3. Admin Login
 4. Exit System

Enter your choice: 1
Enter username: muhib786
Enter password: 123@ned
Enter first name: muhib
Enter last name: qasmi
Enter address: cis dept, neduet, karachi
Enter initial balance: Rs
Minimum balance must be Rs50!
50

User registered successfully!
Press any key to continue . . .
```

```
*****************************************************
          MAIN MENU - LIBRARY SYSTEM
*****************************************************
 1. Register New User
 2. User Login
 3. Admin Login
 4. Exit System

Enter your choice: 2
Username: muhib786
Password: 123@ned

Login successful! Welcome muhib786
Press any key to continue . . .
```

\

[Screenshot 3: Borrowing a book successfully]

```
**************************************************
          USER MENU - WELCOME muhib786
**************************************************
 1. View Available Resources
 2. Borrow a Resource
 3. Return a Resource
 4. Search Library
 5. My Profile Details
 6. Full Borrowing History
 7. Add Balance to Account
 8. Logout

Enter your choice: 2
Enter Resource ID: 1
Borrowed successfully! Due date: 20260518

Book issued successfully!
Press any key to continue . . .
```

6. Task Distribution Among Group Members

Member	Roll No.	Work Done
Syed Muhammad Irtiza Abbas Rizvi	CS-25089	Overall code structure, library class logic, borrowResource and returnResource functions, exception handling, file handling, main menu and user menu
Muhib Ur Rub Qasmi	CS-25086	Resource class, operator overloading, addResource, removeResource, updateResource functions, admin menu
Syed Zayan Mehmood	CS-25090	BorrowRecord class, fine calculation, date helper functions, printIssuedOverdueReport function
Muhammad Hammad Ul Saboor	CS-25080	Person, User, Admin class design, input validation functions, registration and login logic, README and report writing

Note: All group members helped each other with testing and debugging. The table above shows the main tasks each person was responsible for.

7. Conclusion

This project helped us understand how OOP works in a real program. We learned that classes help organize code, inheritance removes repeated code, and encapsulation protects data from wrong changes. Exception handling made our program much more stable. File handling was new for us and a bit confusing at first but we figured it out. Overall this was a good project for our level and we learned a lot from building it.